

TRABAJO PRACTICO N.º 3

Optimización asistida por IA sobre Food Store: filtros, planes de ejecución e índices

Materia	Base de Datos II - Unidad 2
Alumno	Nicolás Moron
Proyecto	Food Store
Motor	PostgreSQL 18.6
Fecha de prueba	10/09/2026

Alcance

La práctica se ejecutó sobre la base aislada **foodstore_tp3**. Las mediciones, planes y equivalencias de este informe provienen de PostgreSQL 18.6; no se usó la base del TP2 ni datos de terceros.

Repositorio

https://github.com/Nixus32xD/DB_Nicolas_Moron

Resumen de evidencia

Actividad	Resultado real
Carga masiva	50.003 productos, 20.002 clientes, 200.002 pedidos y 200.001 detalles en 9,059 s.
Consulta 1	11,849 ms a 0,652 ms: 18,17x.
Consulta 2	162,418 ms a 1,638 ms: 99,16x.
Consulta 3	5,317 ms a 0,199 ms: 26,72x.
Equivalencias	Dos pares comprobados con EXCEPT en ambas direcciones: 0 filas.

1. Carga masiva y protocolo de seguridad

Se creó la copia `foodstore\_tp3` desde `schema.sql`, los datos mínimos y las restricciones de TP2. Antes de cargar se generó un respaldo custom con `pg\_dump -Fc`, validado mediante `pg\_restore -l`.

```
BEGIN;
INSERT INTO cliente ... FROM generate_series(1, 20000);
INSERT INTO producto ... FROM generate_series(1, 50000);
INSERT INTO pedido ... FROM generate_series(1, 200000);
INSERT INTO pedido_detalle ... FROM generate_series(1, 200000);
COMMIT;
ANALYZE cliente; ANALYZE producto; ANALYZE pedido; ANALYZE pedido_detalle;
```

Tabla	Filas luego de la carga
categoria	2
cliente	20.002
producto	50.003
pedido	200.002
pedido_detalle	200.001

Decisión sobre IA

Codex cumplió el rol de agente de código: propuso una carga con `generate\_series`; se revisaron claves foráneas, restricciones y distribución de IDs antes de ejecutarla. La carga se limitó a la copia de TP3 y luego se actualizaron estadísticas con `ANALYZE`.

2. Consultas medidas antes y después

Para cada consulta se corrió `EXPLAIN (ANALYZE, BUFFERS)` antes de aplicar cambios, se revisó el nodo costoso y se repitió exactamente la misma consulta después del índice.

2.1 Comparación de planes

Consulta	Antes	Índice aceptado	Después	Mejora
Productos por categoría y precio	Bitmap Heap Scan + Sort; 11.849 ms	(categoria_id, precio DESC) WHERE activo	Index Scan; 0.652 ms	18.17x
Pedidos recientes	Parallel Seq Scan + Gather Merge; 162.418 ms	pedido(fecha DESC)	Index Scan; 1.638 ms	99.16x
Búsqueda por prefijo	Seq Scan + Sort; 5.317 ms	producto(nombre text_pattern_ops)	Index Scan; 0.199 ms	26.72x

Consulta 1 - productos de una categoría

```
SELECT id, nombre, precio FROM producto
WHERE categoria_id = 1 AND activo = TRUE
AND precio BETWEEN 1000 AND 3500
ORDER BY precio DESC LIMIT 1000;
```

Antes, el índice existente filtró la categoría pero descartó 11.244 filas por precio y realizó `Sort`. Después, `idx\_tp3\_producto\_categoria\_precio\_activo` convirtió el rango de precio en `Index Cond` y entregó las filas en el orden requerido.

Consulta 2 - pedidos recientes

```
SELECT id, cliente_id, fecha, forma_pago FROM pedido
WHERE fecha >= now() - interval '30 days'
ORDER BY fecha DESC LIMIT 1000;
```

Antes se ejecutó `Parallel Seq Scan` y se descartaron 183.560 filas. El índice descendente de fecha eliminó el recorrido completo y el ordenamiento global.

2.2 Búsqueda por prefijo y lectura crítica

Consulta 3 - prefijo de producto

```
SELECT id, nombre, precio FROM producto
WHERE nombre LIKE 'Producto masivo 499%'
ORDER BY nombre;
```

Antes se recorrieron 50.003 productos y se descartaron 49.892. Con `idx\_tp3\_producto\_nombre\_pattern`, PostgreSQL usó un rango del prefijo en un `Index Scan`; el ordenamiento final quedó sobre solo 111 filas.

Lectura crítica del plan posterior de pedidos

Afirmación de IA	Resultado	Evidencia
El Index Scan evita recorrer toda la tabla para localizar pedidos recientes.	Correcta	El plan anterior tenía Parallel Seq Scan; el posterior usa Index Cond por fecha.
El costo 0.42..389.25 significa que tarda 389,25 ms.	Incorrecta	cost es estimación interna; Execution Time real fue 1,638 ms.
El nodo es un Index Only Scan sin leer la tabla.	Incorrecta	El plan dice Index Scan, no Index Only Scan.

Conclusión de la Parte 3

La IA explicó correctamente el uso del índice y la eliminación del ordenamiento, pero se detectaron dos imprecisiones. La lectura del plan real prevalece sobre la explicación en lenguaje natural.

3. Consultas equivalentes

Se implementaron dos alternativas para cada spec y se compararon con `EXCEPT` en ambas direcciones.

Spec	Versión A	Versión B	Verificación
Categorías activas con cantidad de productos activos, incluso cero.	LEFT JOIN + GROUP BY	Subconsulta correlacionada COUNT	A MINUS B = 0; B MINUS A = 0
Clientes con al menos un pedido en los últimos 30 días, sin repetidos.	EXISTS	INNER JOIN + DISTINCT	A MINUS B = 0; B MINUS A = 0

4. DUIA, competencia y cierre

Uso	Aporte de Codex	Validación
Carga	Generó la propuesta con generate_series.	Conteos reales y ANALYZE en foodstore_tp3.
Índices	Propuso tres índices de alcance mínimo.	EXPLAIN ANALYZE confirmó mejoras reales.
Planes	Explicó el plan posterior por nodos.	Se corrigieron costo vs. tiempo e Index Scan vs. Index Only Scan.
Equivalencia	Propuso variantes con JOIN/subconsulta y EXISTS/JOIN.	EXCEPT devolvió 0 filas en los dos pares.

Competencia entre equipos

La guía exige una consulta lenta común y equipos definidos por la cátedra. Esa consulta no fue provista, por lo que no se inventan participantes ni tiempos. El registro queda preparado en `tp3/competencia.md` para completarlo cuando la actividad sea asignada.

Trazabilidad

```
tp3/sql/01_carga_masiva.sql
tp3/sql/02_consultas_base.sql
tp3/sql/03_indices_optimizacion.sql
tp3/sql/04_consultas_equivalentes.sql
tp3/evidencias/01_carga_masiva.txt
tp3/evidencias/02_planes_antes.txt
tp3/evidencias/03_planes_despues.txt
```

Conclusión final

Los cambios se aceptaron con evidencia: cada índice redujo el tiempo real y cambió el nodo que justificaba la optimización. La entrega conserva scripts, planes completos, DUIA y verificaciones de equivalencia para su defensa oral.

Fin de la entrega.